D20A / 37

EXPERIMENT 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.
Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
   Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (`5001`, `5002`, `5003`).
Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender

- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

6. Chain Replacement

When `/replace_chain` is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

- **Python 3.x**
- **Flask** – Web framework for API endpoints
  ```
  pip install Flask
  ```
- **Requests** – For HTTP communication between nodes
  ```
  pip install requests==2.18.4
  ```
- **Postman** – For testing API requests
- Python Standard Libraries:
  - `datetime`
  - `jsonify`
  - `hashlib`
  - `uuid4`
  - `urlparse`
  - `request`

Procedure and Output:

1. Download `hadcoin_node_5001.py`, `hadcoin_node_5002.py`, `hadcoin_node_5003.py`.
2. Install required packages.
3. Run each node in separate terminals.

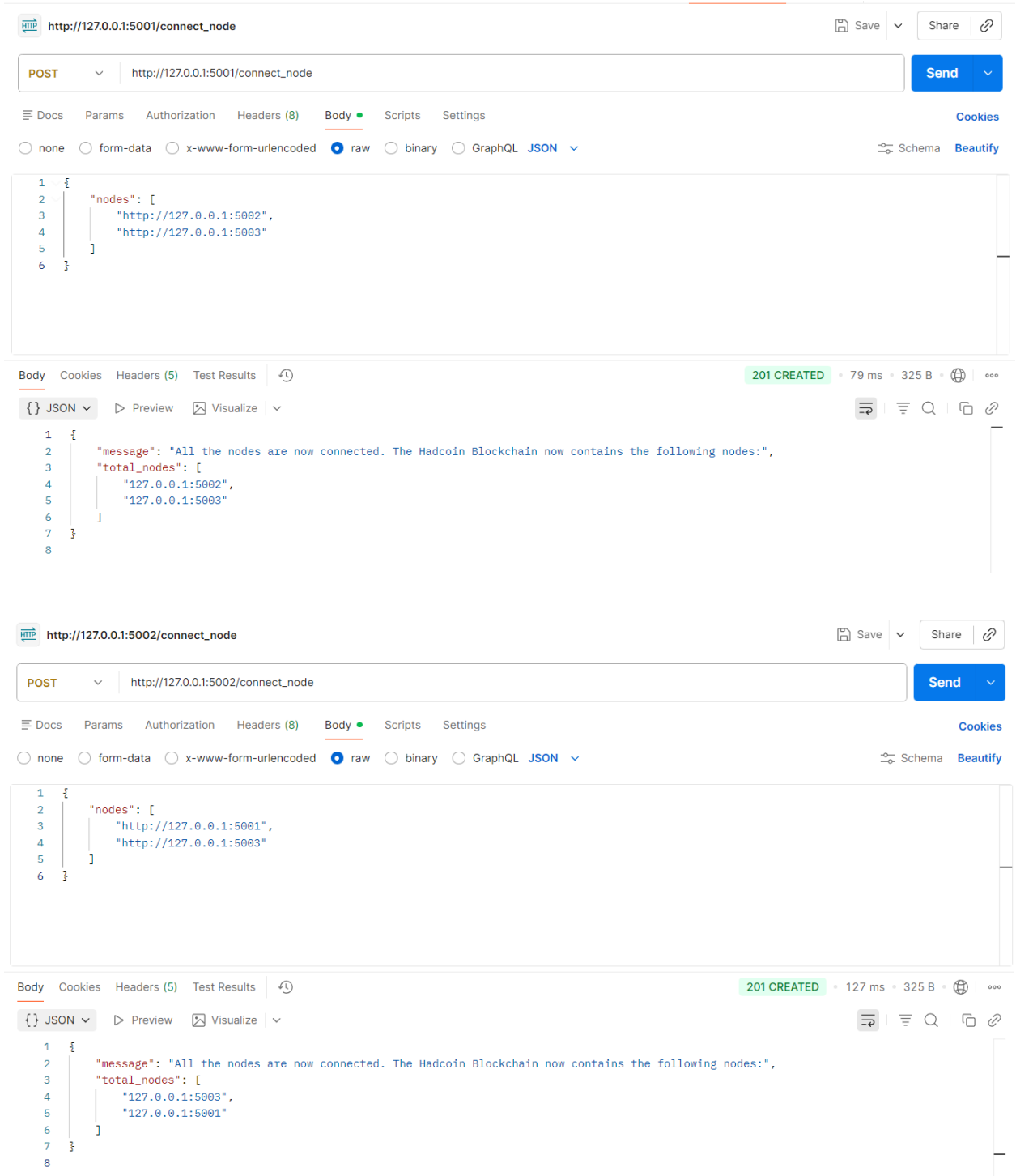
```
PS C:\Lab_3_Blockchain> python -m venv venv
PS C:\Lab_3_Blockchain> venv\Scripts\activate
(venv) PS C:\Lab_3_Blockchain> pip install Flask==0.12.2
Collecting Flask==0.12.2
  Downloading Flask-0.12.2-py2.py3-none-any.whl.metadata (1.9 kB)
Collecting Jinja2>=2.4 (from Flask==0.12.2)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting Werkzeug>=0.7 (from Flask==0.12.2)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting click>=2.0 (from Flask==0.12.2)
  Downloading click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=0.21 (from Flask==0.12.2)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting MarkupSafe>=2.0 (from Jinja2>=2.4->Flask==0.12.2)
  Using cached markupsafe-3.0.3-cp313-cp313-win_amd64.whl.metadata (2.8 kB)
```

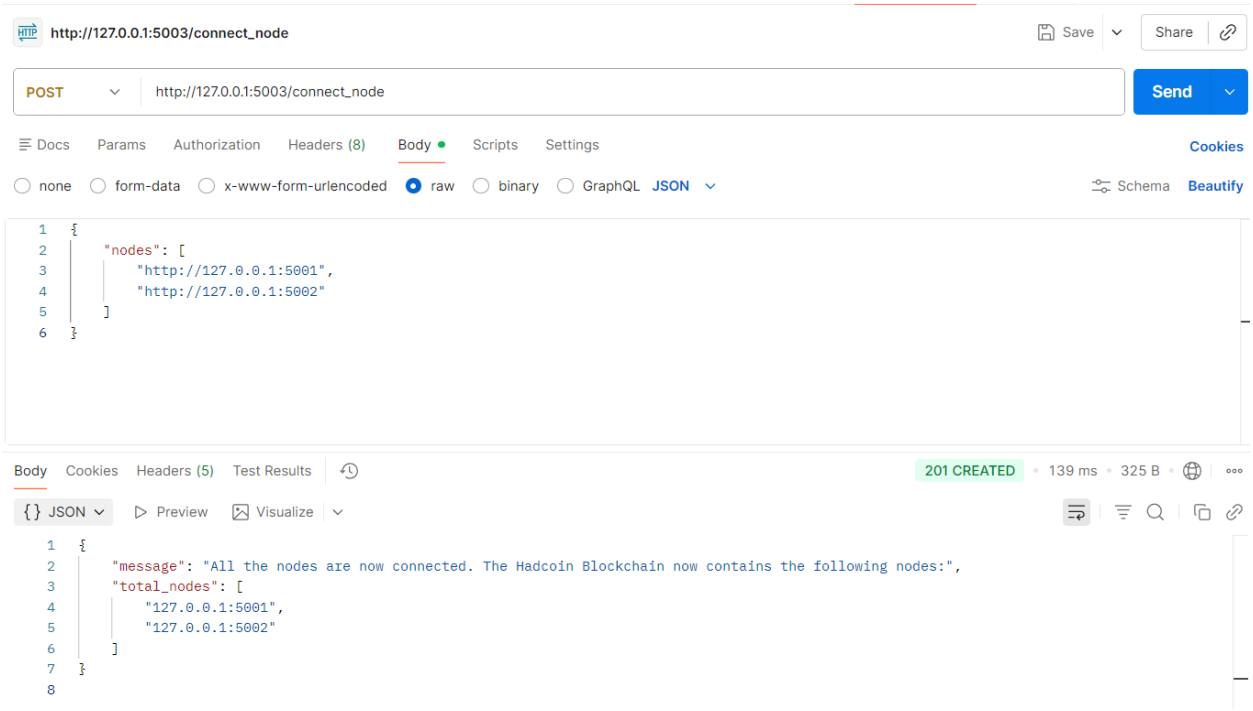
```
(venv) PS C:\Lab_3_Blockchain> python hadcoin_node_5001.py
 * Serving Flask app 'hadcoin_node_5001'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5001
 * Running on http://192.168.0.102:5001
Press CTRL+C to quit
```

```
PS C:\Lab_3_Blockchain> & c:/Lab_3_Blockchain/venv/Scripts/Activate.ps1
(venv) PS C:\Lab_3_Blockchain> python hadcoin_node_5002.py
 * Serving Flask app 'hadcoin_node_5002'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5002
 * Running on http://192.168.0.102:5002
Press CTRL+C to quit
```

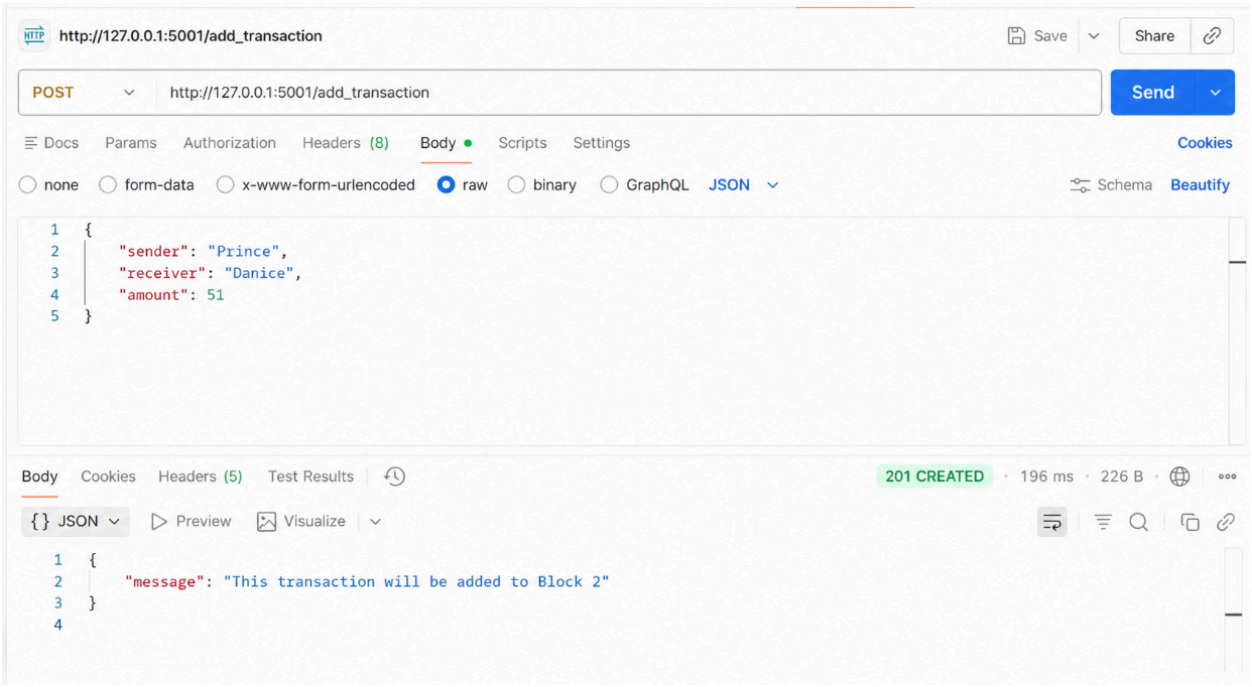
```
Problems  Output  Debug Console  Terminal  Ports
PS C:\Lab_3_Blockchain> & c:/Lab_3_Blockchain/venv/Scripts/Activate.ps1
(venv) PS C:\Lab_3_Blockchain> python hadcoin_node_5003.py
 * Serving Flask app 'hadcoin_node_5003'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5003
 * Running on http://192.168.0.102:5003
Press CTRL+C to quit
```

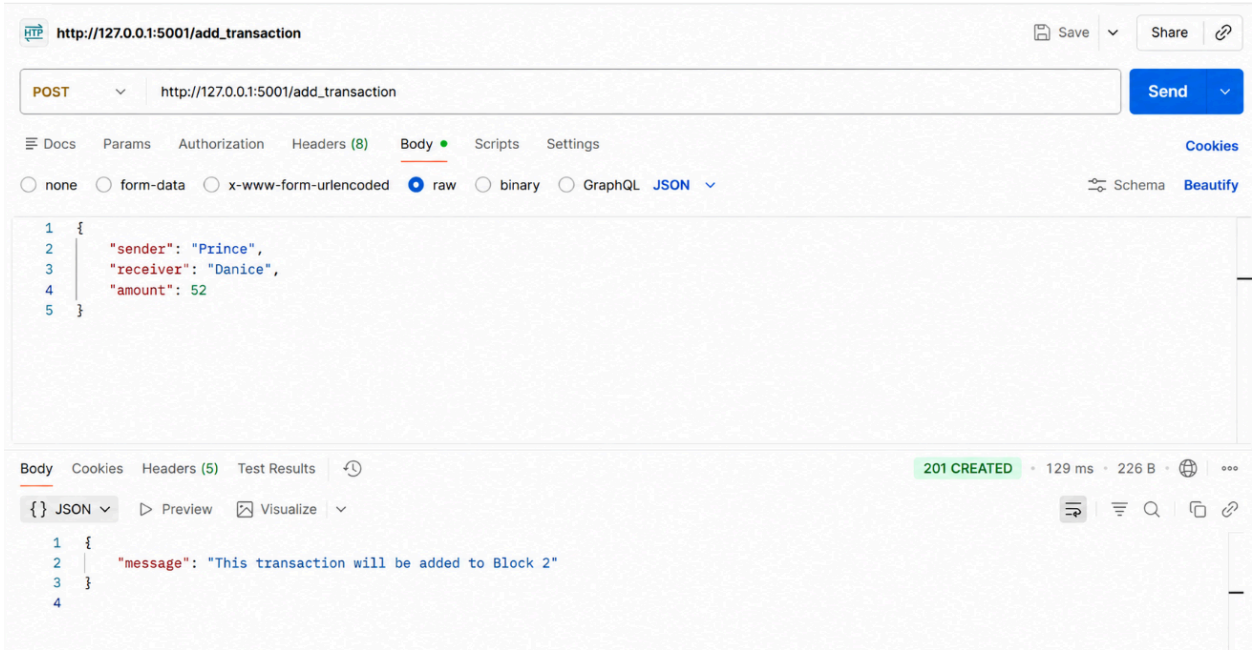
4. Connect nodes using Postman – POST `/connect_node`.

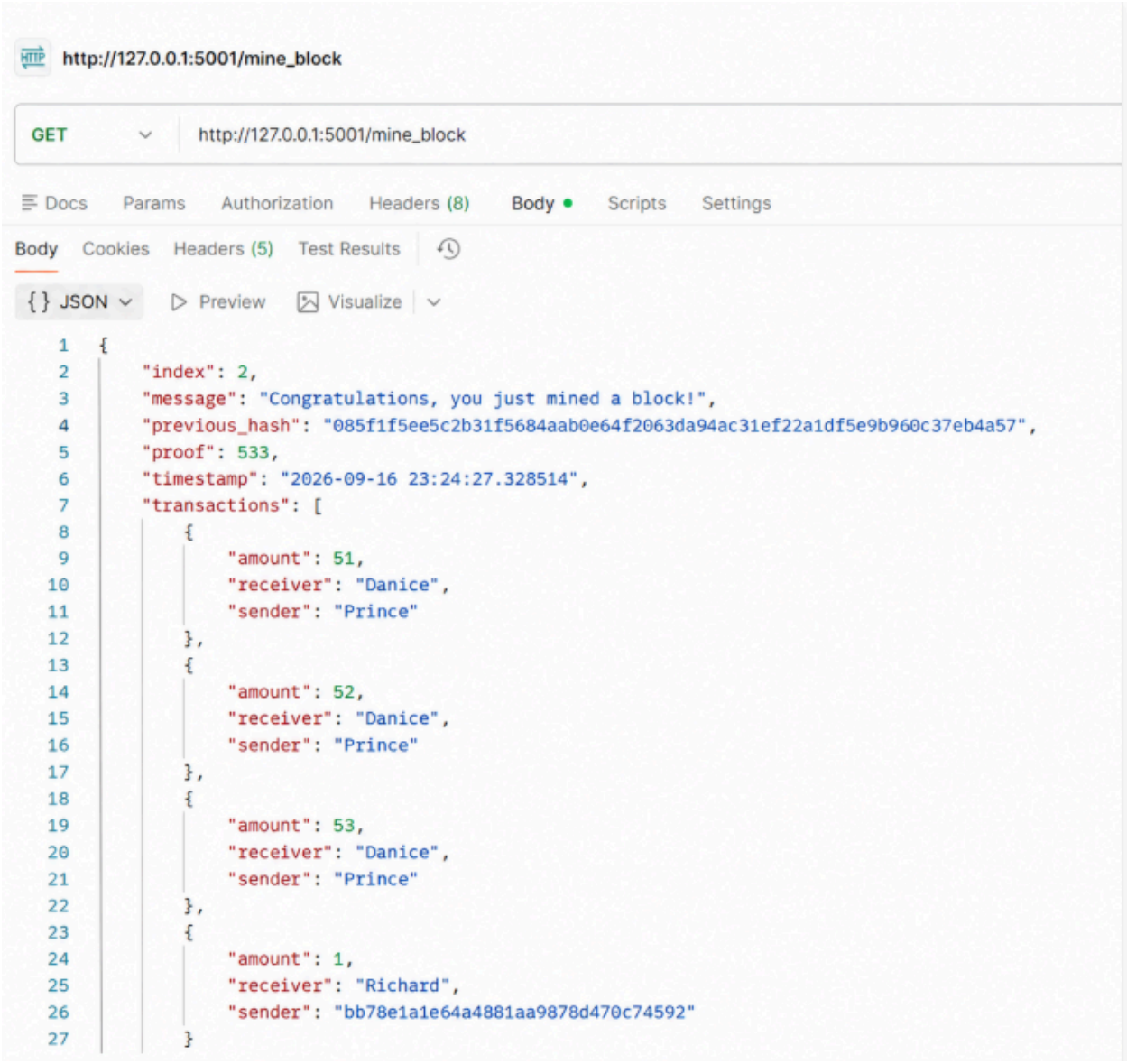

http://127.0.0.1:5001/connect_node
Save
Share
POST
http://127.0.0.1:5001/connect_node
Send
Docs
Params
Authorization
Headers (8)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"nodes": [
"http://127.0.0.1:5002",
"http://127.0.0.1:5003"
]
}
Body
Cookies
Headers (5)
Test Results
201 CREATED
79 ms
325 B
{} JSON
Preview
Visualize
{
"message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
"total_nodes": [
"127.0.0.1:5002",
"127.0.0.1:5003"
]
}
http://127.0.0.1:5002/connect_node
Save
Share
POST
http://127.0.0.1:5002/connect_node
Send
Docs
Params
Authorization
Headers (8)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"nodes": [
"http://127.0.0.1:5001",
"http://127.0.0.1:5003"
]
}
Body
Cookies
Headers (5)
Test Results
201 CREATED
127 ms
325 B
{} JSON
Preview
Visualize
{
"message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:",
"total_nodes": [
"127.0.0.1:5003",
"127.0.0.1:5001"
]
}



5. Add transactions – POST




http://127.0.0.1:5001/add_transaction
Save
Share
POST
http://127.0.0.1:5001/add_transaction
Send
Docs
Params
Authorization
Headers (8)
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"sender": "Prince",
"receiver": "Danice",
"amount": 52
}
Body
Cookies
Headers (5)
Test Results
201 CREATED
129 ms
226 B
JSON
Preview
Visualize
{
"message": "This transaction will be added to Block 2"
}



6. Each node must have **different chain lengths**

http://127.0.0.1:5001/get_chain

GET http://127.0.0.1:5001/get_chain

Docs Params Authorization Headers (8) Body Scripts Settings

Body Cookies Headers (5) Test Results

{} JSON Preview Visualize


```
            "transactions": [
                {
                    "amount": 51,
                    "receiver": "Danice",
                    "sender": "Prince"
                },
                {
                    "amount": 52,
                    "receiver": "Danice",
                    "sender": "Prince"
                },
                {
                    "amount": 53,
                    "receiver": "Danice",
                    "sender": "Prince"
                },
                {
                    "amount": 1,
                    "receiver": "Richard",
                    "sender": "bb78e1a1e64a4881aa9878d470c74592"
                }
            ]
        }
    ],
    "length": 2
}
```


http://127.0.0.1:5002/get_chain

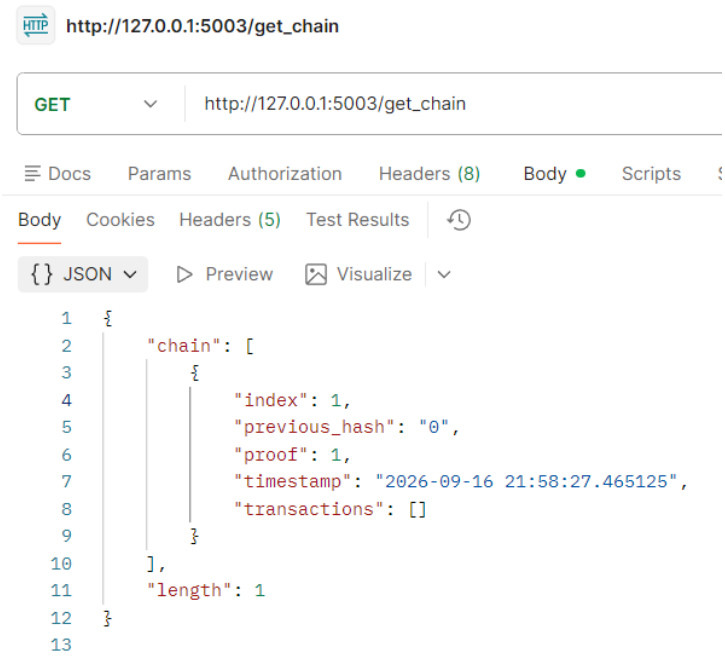
GET http://127.0.0.1:5002/get_chain

Docs Params Authorization Headers (8) Body Scripts Settin

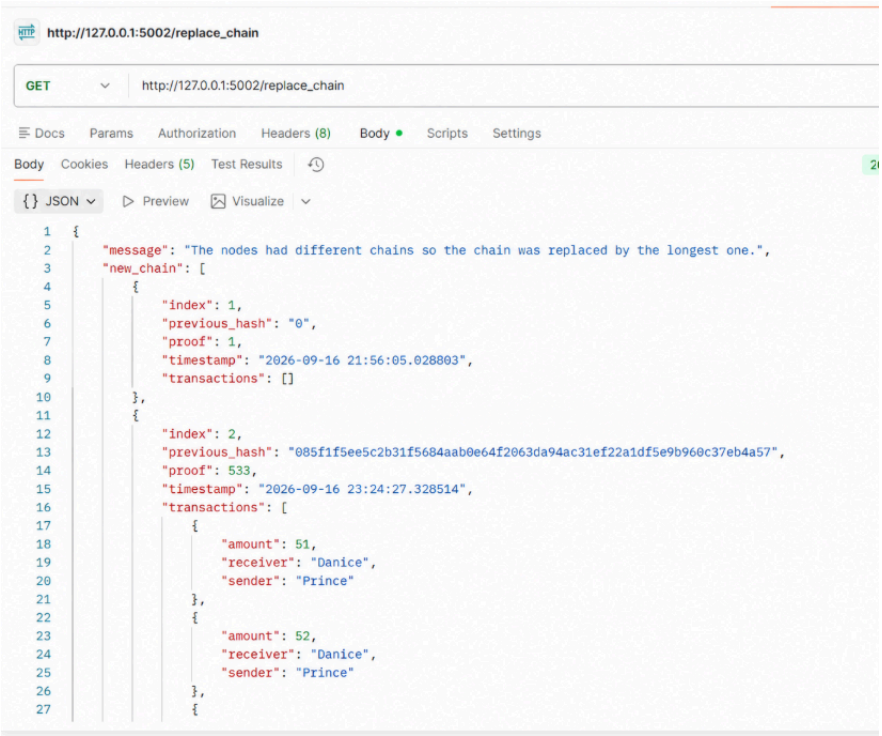
Body Cookies Headers (5) Test Results

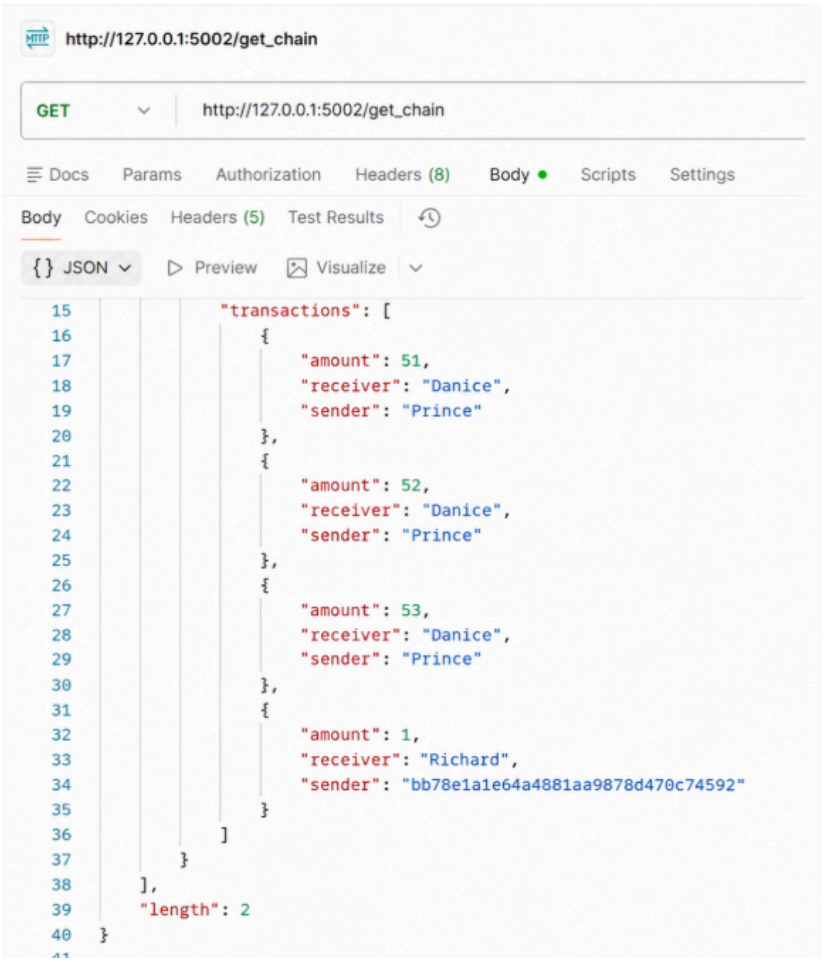
{} JSON Preview Visualize

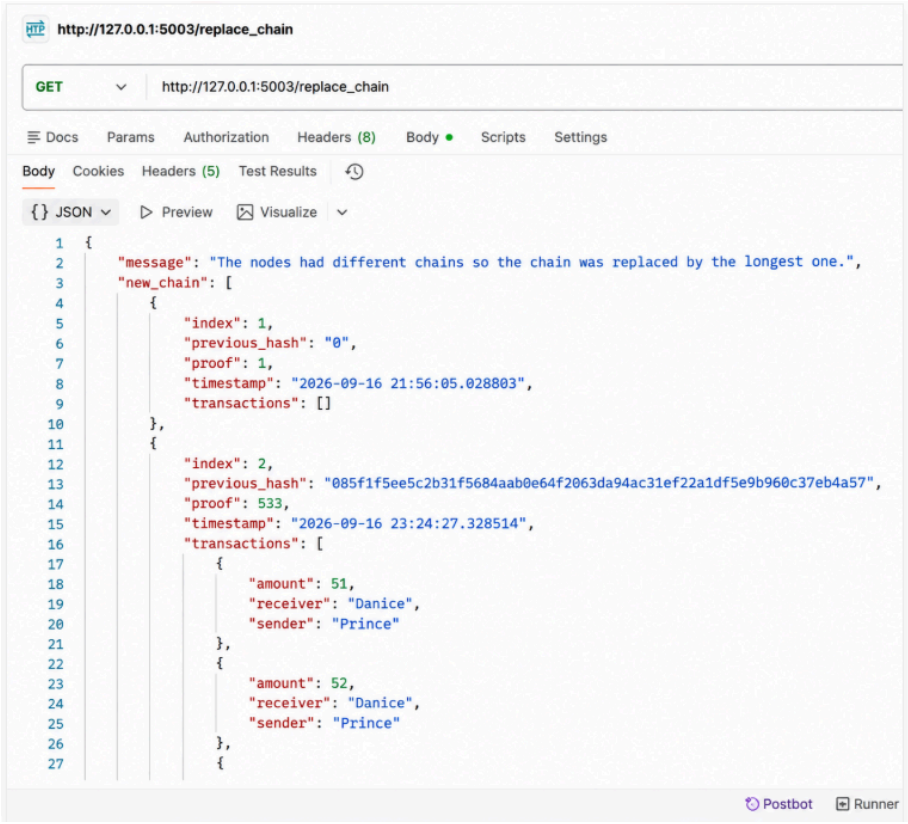

```
{
    "chain": [
        {
            "index": 1,
            "previous_hash": "0",
            "proof": 1,
            "timestamp": "2026-09-16 21:57:58.075726",
            "transactions": []
        }
    ],
    "length": 1
}
```



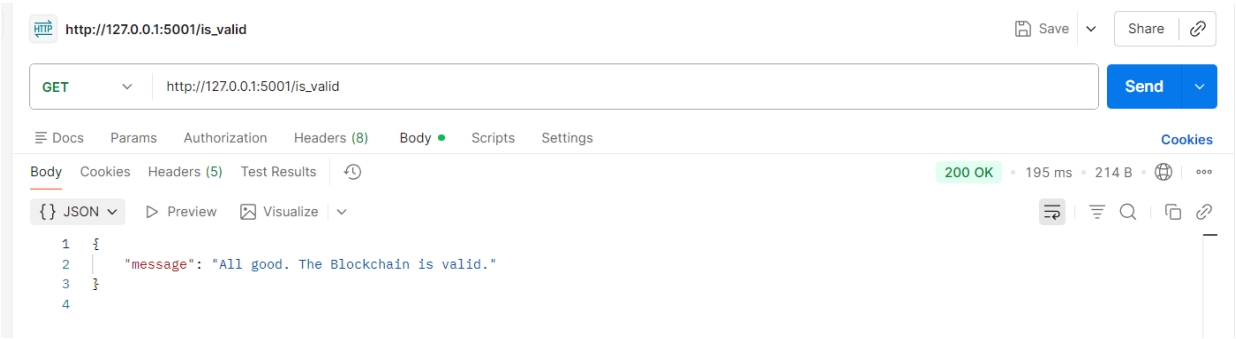
7. Replacing the Chain Using Longest Chain Rule – Node 5002 and 5003

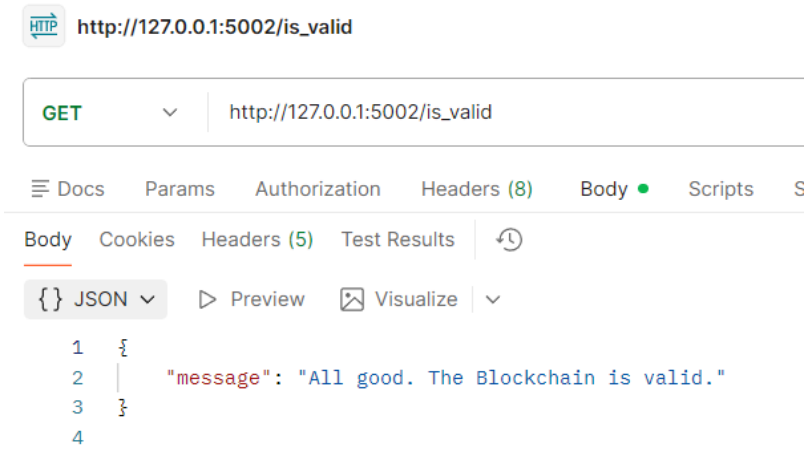


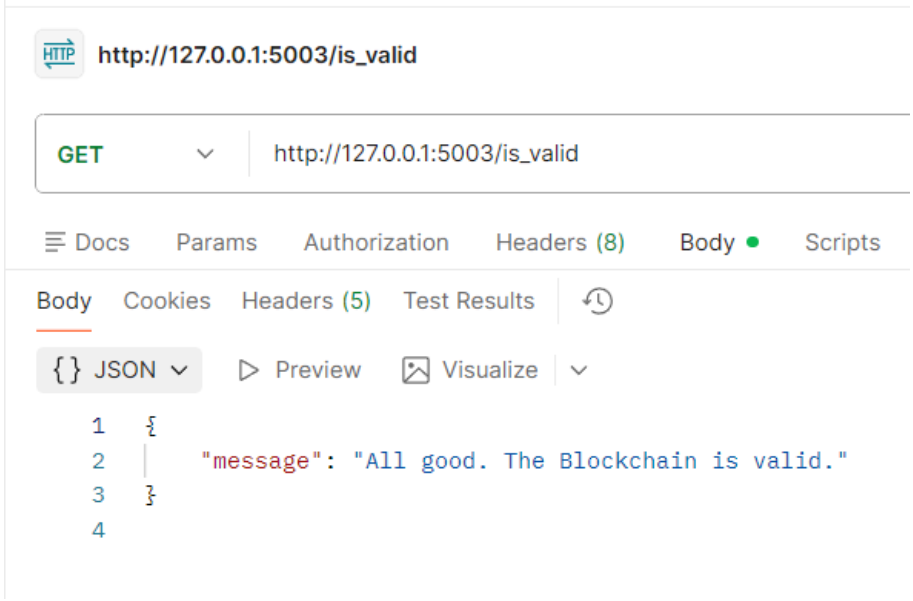

http://127.0.0.1:5002/get_chain
GET
http://127.0.0.1:5002/get_chain
Docs
Params
Authorization
Headers (8)
Body
Scripts
Settings
Body
Cookies
Headers (5)
Test Results
{ } JSON
Preview
Visualize
"transactions": [
{
"amount": 51,
"receiver": "Danice",
"sender": "Prince"
},
{
"amount": 52,
"receiver": "Danice",
"sender": "Prince"
},
{
"amount": 53,
"receiver": "Danice",
"sender": "Prince"
},
{
"amount": 1,
"receiver": "Richard",
"sender": "bb78e1a1e64a4881aa9878d470c74592"
}
]
}
],
"length": 2
}



8. Validation of Blockchain







8. Removing Transactions After Adding Them to a Block

hadcoin.py > ...

```python
class Blockchain:

    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                 'timestamp': str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash': previous_hash,
                 'transactions': self.transactions}                    # Adding transactions to make the
        self.transactions = []                                         # The list of transacction should
        self.chain.append(block)
        return block

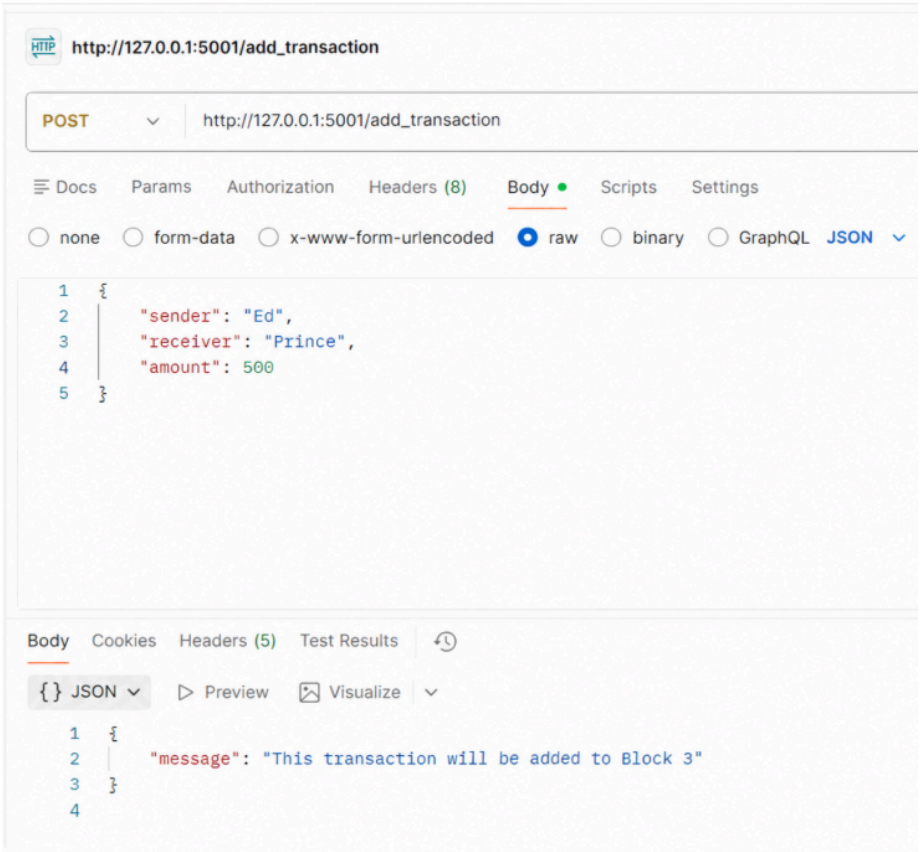
    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
            hash_operation = hashlib.sha256(str(new_proof**2 - previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

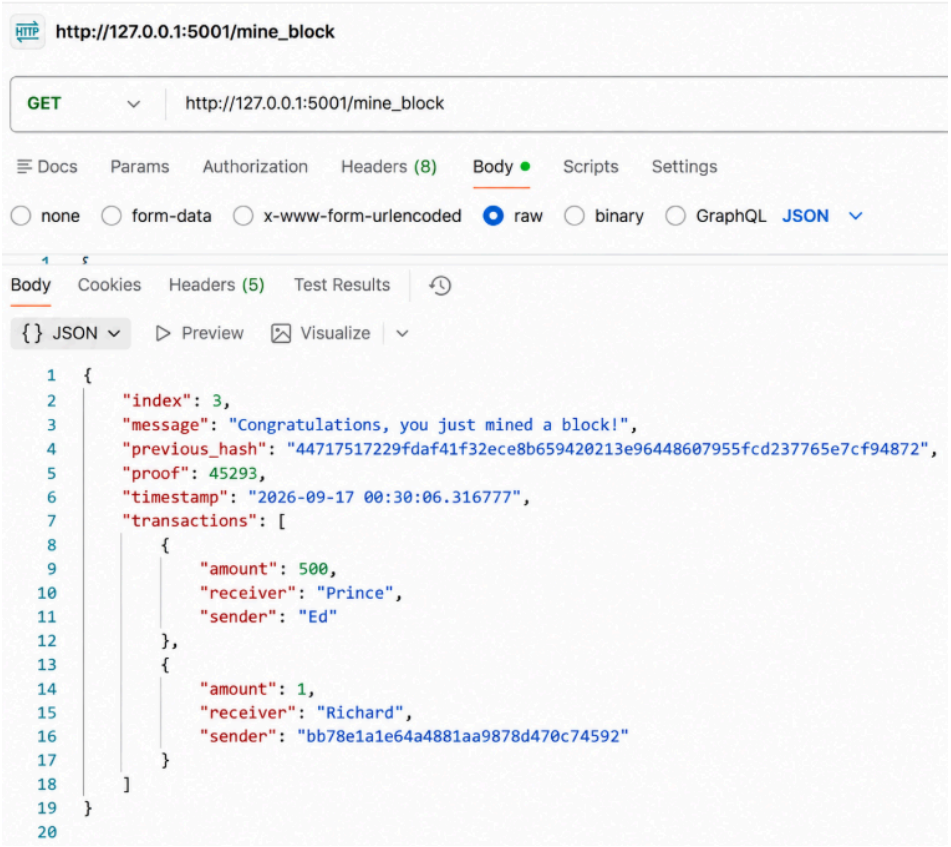
    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys = True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
```

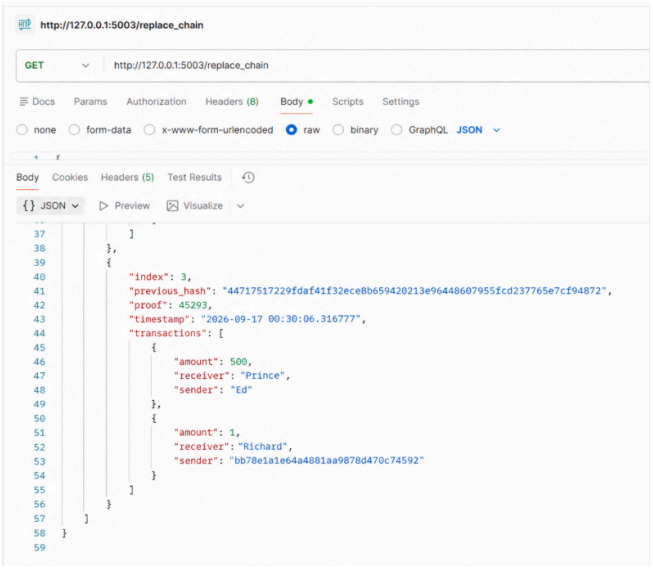
9.Adding a New Transaction After Mining

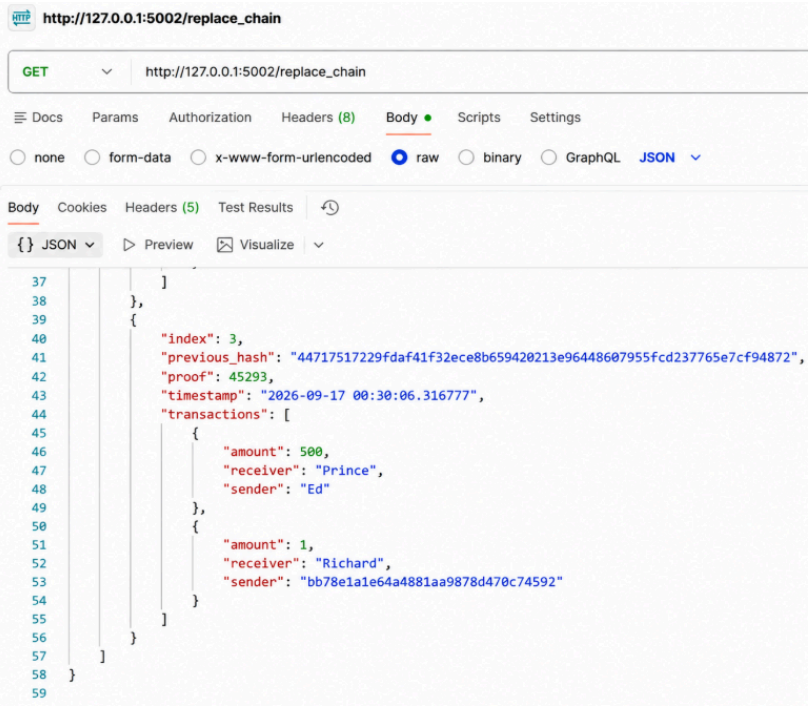


10. Mining the Second Block



11. Final blockchain synchronization





Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.